Artificial Intelligence Project Report

Garbage Image Classification

Using EfficientNet-B6



Project Type:	Deep Learning / Image Classification
Framework:	PyTorch / Torchvision
Dataset:	Garbage Classification V2
Model:	ImageNet-pretrained EfficientNet-B6
Final Test Accuracy:	**94.18%**
Deployment:	Gradio

October 5, 2026

Contents

1 Abstract **2**

2 Introduction **2**

3 Project Objectives **3**

4 Dataset Description **3**
4.1 Class Distribution 3
4.2 Image Resolution Analysis 4

5 Data Preprocessing **5**
5.1 Dataset Cleaning 5
5.2 Image Transformations 5
5.3 Train, Validation and Test Split 7

6 Model Selection: EfficientNet-B6 **8**

7 Model Training **9**

8 Fine-Tuning **9**

9 Model Saving **10**

10 Final Test Evaluation **10**

11 Confusion Matrix **10**

12 Classification Report **11**

13 Deployment **12**

14 System Workflow **13**

15 Discussion **13**

16 Limitations **13**

17 Conclusion **14**

A Selected Core Code **15**
A.1 Dataset Inspection 15
A.2 Image Transformations 15
A.3 EfficientNet-B6 Initialization 16
A.4 Training Loop 16
A.5 Fine-Tuning 17
A.6 Final Test Evaluation 17
A.7 Classification Report 17
A.8 Gradio Prediction Interface 18

B Complete Notebook Source **18**

C Reproducibility Notes **18**

1 Abstract

This project develops an image classification system for automatically recognizing different types of garbage. The notebook uses the Garbage Classification V2 dataset and builds a ten-class image classifier using an ImageNet-pretrained EfficientNet-B6 model. The workflow includes dataset inspection, image cleaning, data augmentation, normalization, a 70/15/15 train-validation-test split, transfer learning, fine-tuning, quantitative evaluation, confusion-matrix analysis, and a Gradio-based deployment interface.

The initial three-epoch training phase achieved 84.47% training accuracy and 84.61% validation accuracy. After two additional fine-tuning epochs with a lower learning rate, the training accuracy increased to 94.87% and validation accuracy to 94.18%. The final evaluation on the held-out test set produced **94.18% accuracy** over 1,839 test images. The weighted F1-score was 0.9421, demonstrating strong overall classification performance.

2 Introduction

Waste sorting is an important part of effective waste management and recycling. Manual identification of garbage categories can be time-consuming and inconsistent. Computer vision and deep learning can assist this process by learning visual patterns from labeled waste images and predicting the category of a new image.

The objective of this project is to construct a multi-class image classifier that distinguishes ten garbage categories:

- battery
- biological
- cardboard
- clothes
- glass
- metal
- paper
- plastic
- shoes
- trash

The supplied notebook implements the complete experimental pipeline from data acquisition to model deployment.

3 Project Objectives

The main objectives are:

1. Inspect and understand the garbage image dataset.
2. Remove unsupported or corrupted image files.
3. Apply image resizing, augmentation, tensor conversion, and normalization.
4. Split the data into training, validation, and testing subsets.
5. Use transfer learning with EfficientNet-B6.
6. Fine-tune the trained network for improved performance.
7. Evaluate the final model using accuracy, precision, recall, F1-score, and a confusion matrix.
8. Save the trained weights and full model.
9. Provide an image-classification interface using Gradio.

4 Dataset Description

The notebook downloads the *garbage-classification-v2* dataset and works with its `original` image directory. The dataset contains 12,259 images distributed among ten categories.

4.1 Class Distribution

Table 1 summarizes the number of images found in each category.

Table 1: Dataset class distribution

Category	**Images**
clothes	1892
glass	1736
plastic	1597
shoes	1449
cardboard	1411
paper	1336
metal	930
battery	756
biological	699
trash	453
Total	**12259**

The class distribution is not perfectly balanced. Clothes has the largest number of images (1,892), while trash has the smallest number (453). This imbalance is relevant when interpreting class-level metrics.

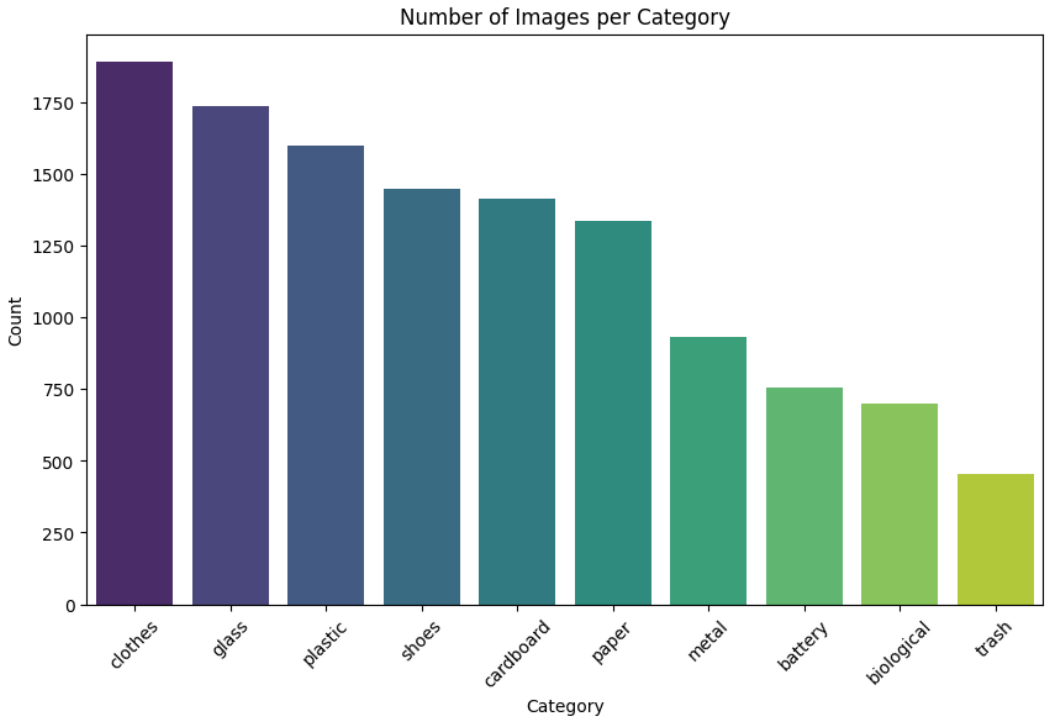


Figure 1: Number of images per garbage category

4.2 Image Resolution Analysis

The notebook examines a sample of 500 images and reports an average width of 424.99 pixels and an average height of 514.49 pixels.

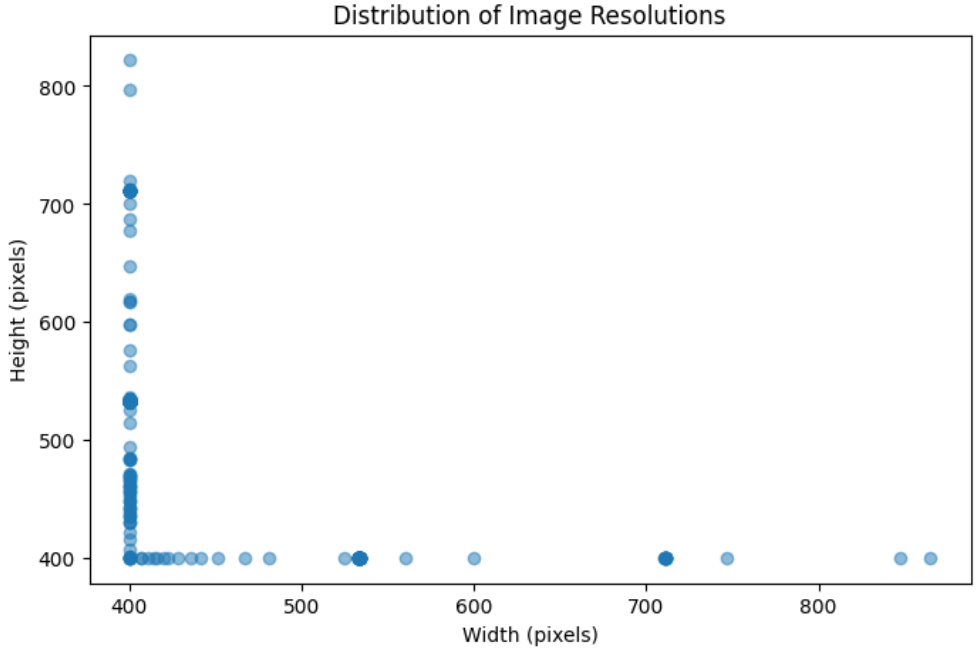


Figure 2: Distribution of sampled image resolutions

A random sample of ten original images is also visualized to verify that the dataset contains diverse garbage objects and appearances.



Figure 3: Random sample of garbage images from the dataset

5 Data Preprocessing

5.1 Dataset Cleaning

The notebook traverses the dataset directory and removes files that are not PNG/JPG/JPEG images. Each supported image is opened and verified using PIL. If an image cannot be verified, it is deleted as a corrupted file.

The notebook reports:

Dataset cleaning completed.

5.2 Image Transformations

For training, the following transformations are applied:

1. Resize to 224×224 pixels.
2. Random horizontal flip.
3. Random vertical flip.
4. Random rotation up to 30 degrees.
5. Color jitter for brightness and contrast.
6. Conversion to tensor.
7. ImageNet mean/std normalization.

For validation and testing, only deterministic resizing, tensor conversion, and ImageNet normalization are applied.

The resulting tensor shape is:

```
torch.Size([3,224,224])
```

and the standardized image resolution is 224×224.

The training augmentation process is illustrated below. The example shows how horizontal flipping, rotation, and color jitter can produce varied training inputs from the same original image.

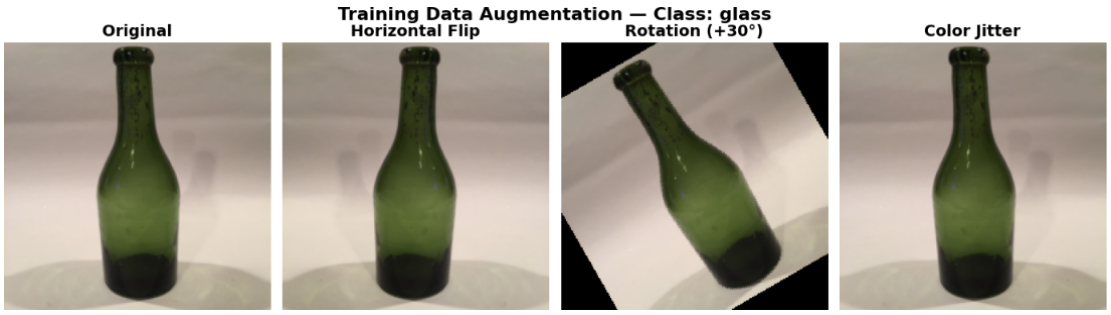


Figure 4: Example of training data augmentation applied to a glass-class image.

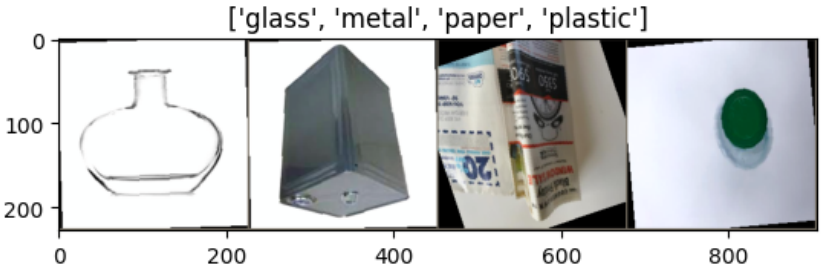


Figure 5: Example batch after training transformations

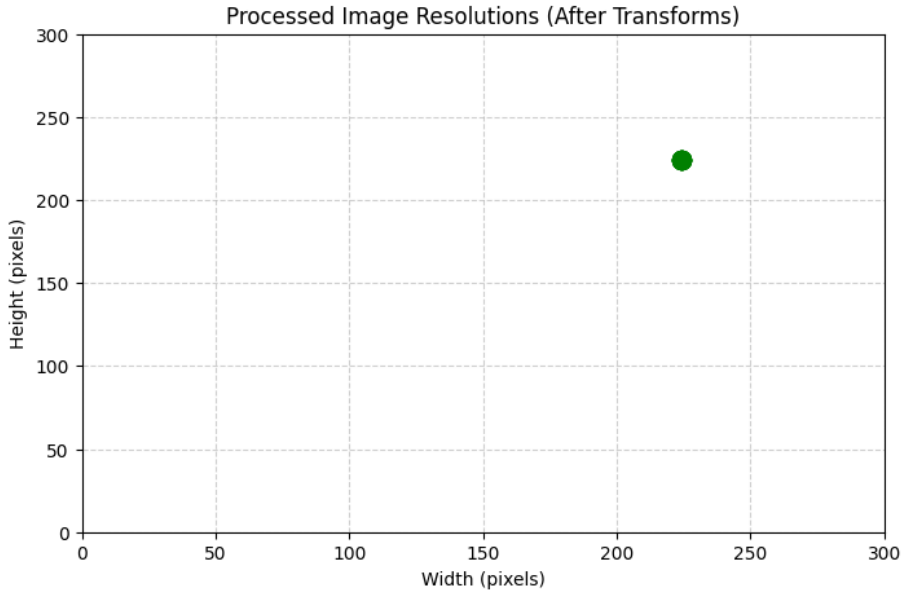


Figure 6: Processed image resolution after transformations

5.3 Train, Validation and Test Split

The notebook creates a 70/15/15 split:

Table 2: Dataset split

Subset	**Percentage**	**Images**
Training	70%	8581
Validation	15%	1839
Testing	15%	1839
Total	100%	12259

The DataLoader batch size is 16. Training data are shuffled, while validation and test data are not shuffled.

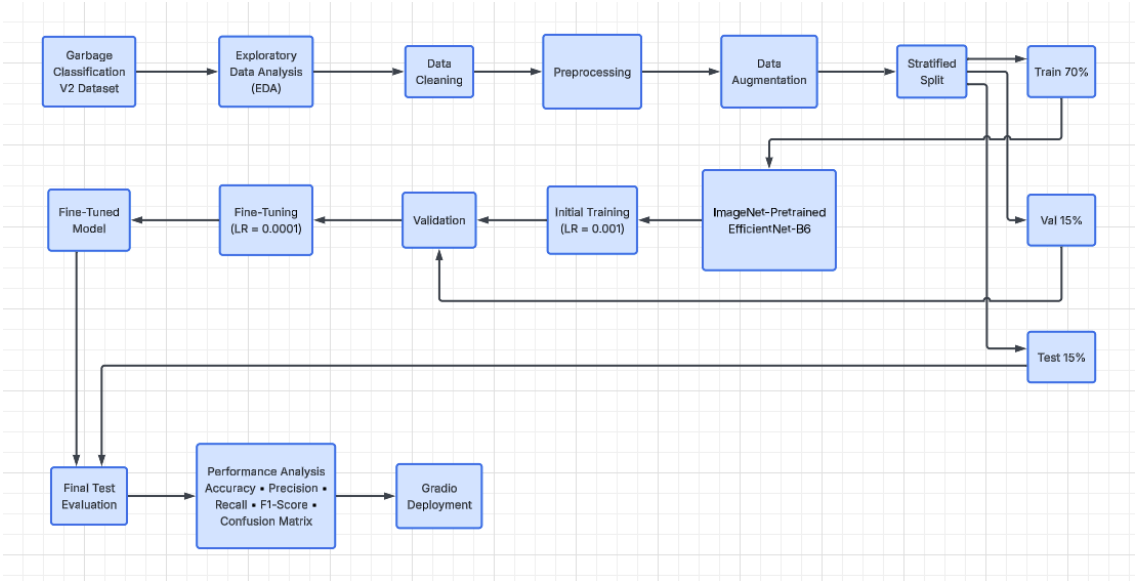


Figure 7: End-to-end workflow of the proposed EfficientNet-B6 garbage classification system.

6 Model Selection: EfficientNet-B6

The project uses EfficientNet-B6 with ImageNet-pretrained weights. Transfer learning is appropriate because a pretrained convolutional neural network has already learned useful visual representations such as edges, textures, shapes, and object patterns.

The original ImageNet classifier is replaced with a new linear layer containing ten output neurons, corresponding to the ten garbage categories.

The transfer-learning architecture used in the project is illustrated below.

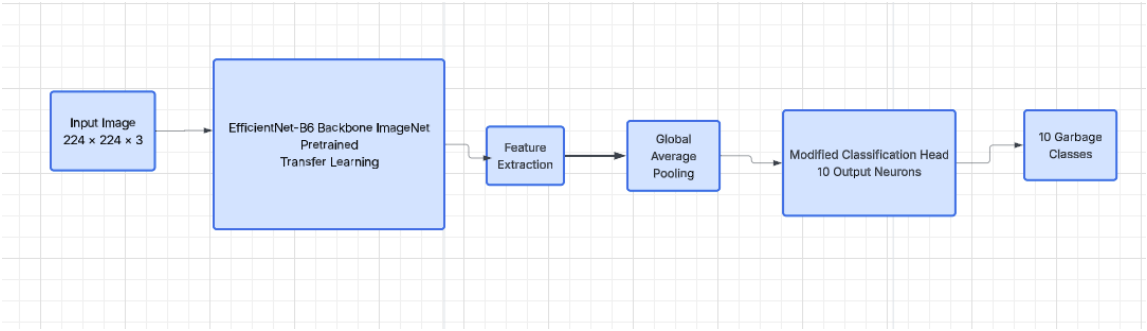


Figure 8: Transfer-learning architecture using ImageNet-pretrained EfficientNet-B6 for 10-class garbage classification.

The notebook confirms:

```
Using device: cuda
GPU: Tesla T4
EfficientNet-B6 initialized successfully.
Number of output classes: 10
```

7 Model Training

The training function uses a standard supervised learning loop:

1. Move a batch to the selected device.
2. Clear optimizer gradients.
3. Perform a forward pass.
4. Calculate cross-entropy loss.
5. Backpropagate the loss.
6. Update model parameters.
7. Calculate training accuracy.

The first training phase uses three epochs.

Table 3: Initial EfficientNet-B6 training

Epoch	Loss	Training Accuracy
1	0.8853	71.98%
2	0.5790	81.72%
3	0.4814	84.47%

After the initial phase, the validation accuracy was 84.61%.

8 Fine-Tuning

The trained EfficientNet-B6 model is fine-tuned using the Adam optimizer with a lower learning rate of 0.0001. Two additional epochs are performed.

Table 4: Fine-tuning results

Epoch	Loss	Training Accuracy
1	0.2556	91.56%
2	0.1591	94.87%

After fine-tuning, validation accuracy increased from 84.61% to **94.18%**. This is a substantial improvement over the initial model.

9 Model Saving

The notebook saves several forms of the trained model:

- `garbage_eff_b6.pth`: initial trained weights.
- `garbage_eff_b6_finetuned.pth`: fine-tuned weights.
- `garbage_full_eff_b6.pth`: full model object containing structure and weights.

The notebook reports successful saving of the fine-tuned weights and full model.

10 Final Test Evaluation

The final model is evaluated using the held-out test set of 1,839 images.

Final Test Accuracy: 94.18%

The same 94.18% value is obtained for the final test accuracy and the post-fine-tuning validation accuracy.

11 Confusion Matrix

The confusion matrix compares the true garbage categories with the categories predicted by EfficientNet-B6.

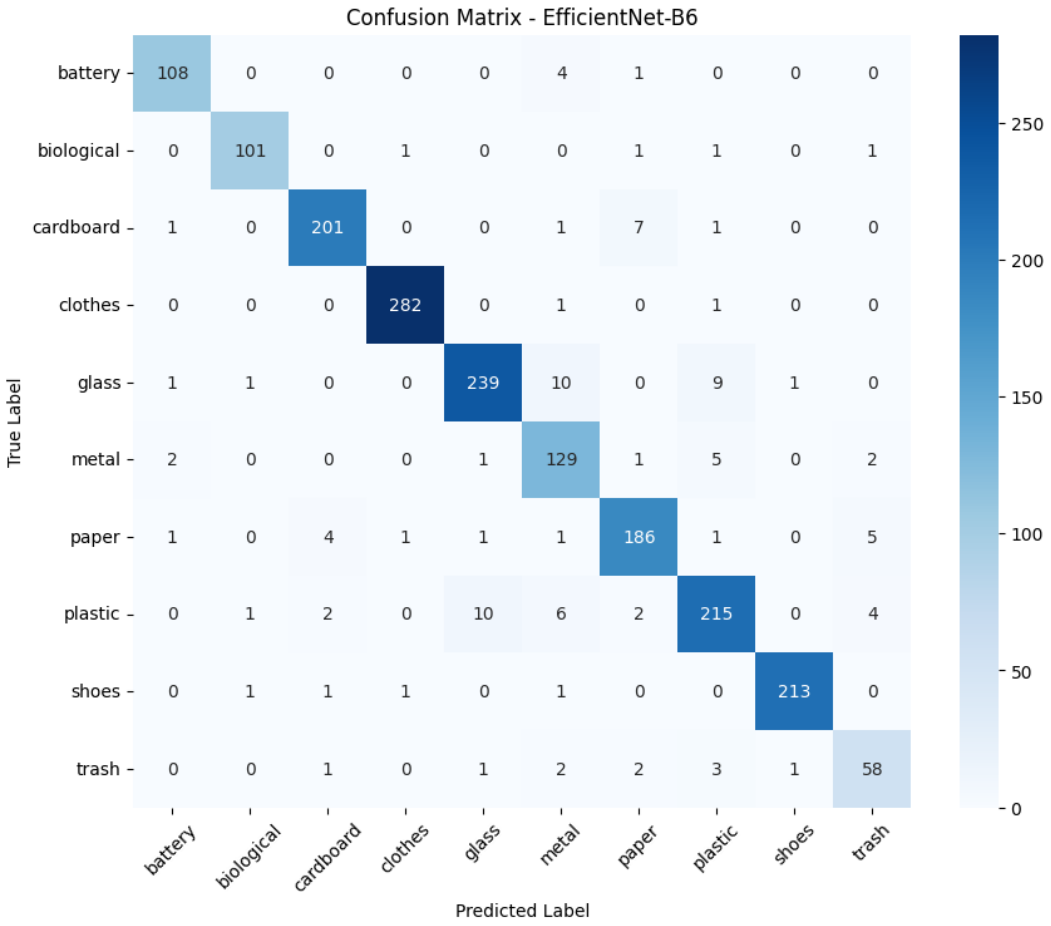


Figure 9: Confusion matrix for EfficientNet-B6 on the test set

The confusion matrix provides class-level information that cannot be obtained from accuracy alone. It can be used to identify which categories are most frequently confused with one another.

12 Classification Report

The final classification report from the notebook is reproduced below.

Table 5: Class-wise test-set performance

Class	Precision	Recall	F1	Support
battery	0.9558	0.9558	0.9558	113
biological	0.9712	0.9619	0.9665	105
cardboard	0.9617	0.9526	0.9571	211
clothes	0.9895	0.9930	0.9912	284
glass	0.9484	0.9157	0.9318	261
metal	0.8323	0.9214	0.8746	140
paper	0.9300	0.9300	0.9300	200
plastic	0.9110	0.8958	0.9034	240
shoes	0.9907	0.9816	0.9861	217
trash	0.8286	0.8529	0.8406	68
Accuracy			**0.9418**	1839
Macro Avg	0.9319	0.9361	0.9337	1839
Weighted Avg	0.9429	0.9418	0.9421	1839

The strongest F1-score is obtained for *clothes* (0.9912), followed by *shoes* (0.9861). The lowest F1-score is observed for *trash* (0.8406), followed by *metal* (0.8746). The weighted average F1-score is 0.9421.

13 Deployment

The notebook prepares a deployment version using Gradio. The fine-tuned model weights are loaded into a new EfficientNet-B6 architecture. A prediction function converts an uploaded image to RGB, applies the same inference preprocessing, performs model inference, and returns the predicted garbage class.

The deployment uses the fine-tuned weight file:

```
garbage_eff_b6_finetuned.pth
```

The Gradio interface is launched from the notebook. The notebook output indicates that the Gradio cell starts an interface suitable for interactive image classification.

A representative Gradio prediction interface from the presentation is shown below. It demonstrates uploading a garbage image and displaying the predicted class.

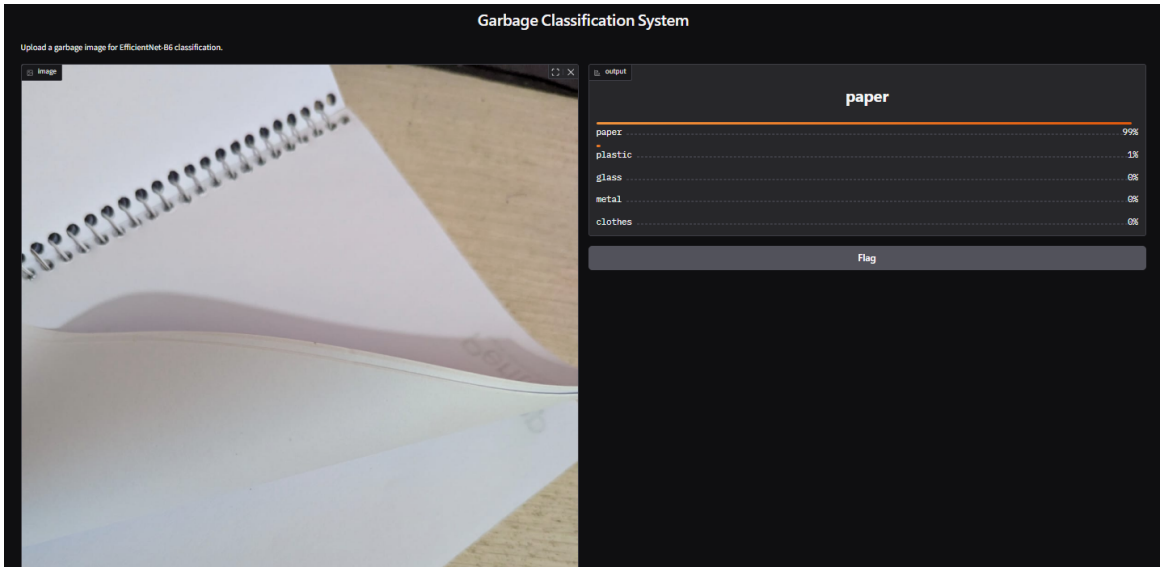


Figure 10: Example Gradio deployment interface showing a paper-class prediction.

14 System Workflow

The overall system can be summarized as:

Dataset Acquisition → Cleaning → Augmentation → Train/Val/Test Split → EfficientNet-B6 → Training → Fine-Tuning → Evaluation → Gradio Deployment

15 Discussion

The results show that transfer learning with EfficientNet-B6 performs well on the ten-class garbage classification task. The initial model reached 84.61% validation accuracy, while fine-tuning increased validation accuracy to 94.18%. The final test accuracy was also 94.18%.

The class-level results indicate that visually distinctive categories such as clothes and shoes are classified particularly well. Classes such as trash and metal have lower F1-scores, suggesting that these categories may contain greater visual variability or may share visual characteristics with other categories.

The dataset is also imbalanced. In particular, the trash class contains only 453 images, compared with 1,892 clothes images. This imbalance should be considered when interpreting overall accuracy.

16 Limitations

The report is based strictly on the supplied notebook. The notebook uses a three-epoch initial training phase followed by two fine-tuning epochs. Therefore, the experiment does not provide a systematic comparison of multiple architectures, extensive hyperparameter search, or repeated cross-validation.

The notebook also contains a Drive-access cell that reports that the expected shared folder was not found. This did not prevent the final evaluation because the trained model was already available in the notebook runtime.

The Gradio deployment cell is included in the notebook, but the supplied notebook does not provide a persistent public deployment URL in its saved output.

17 Conclusion

This project successfully implements an end-to-end deep learning pipeline for garbage image classification. The system uses data cleaning and augmentation, a 70/15/15 dataset split, ImageNet-pretrained EfficientNet-B6, fine-tuning, and detailed evaluation.

The most important result is the final test accuracy of **94.18%** on 1,839 held-out test images. The weighted F1-score of **0.9421** further indicates strong overall performance across the ten garbage categories.

The project demonstrates how transfer learning can be applied to a practical computer vision problem and how a trained model can be prepared for interactive use through Gradio.

A Selected Core Code

The following code fragments represent the main implementation stages.

A.1 Dataset Inspection

```
data_dir = '/content/garbage-classification/original'

categories = [
    d for d in os.listdir(data_dir)
    if os.path.isdir(os.path.join(data_dir, d))
]

data_counts = []
for cat in categories:
    count = len(os.listdir(os.path.join(data_dir, cat)))
    data_counts.append({
        'Category': cat,
        'Count': count
    })

df = pd.DataFrame(data_counts).sort_values(
    by='Count',
    ascending=False
)
```

A.2 Image Transformations

```
data_transforms = {
    'train': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.RandomHorizontalFlip(),
        transforms.RandomVerticalFlip(),
        transforms.RandomRotation(30),
        transforms.ColorJitter(
            brightness=0.2,
            contrast=0.2
        ),
        transforms.ToTensor(),
        transforms.Normalize(
            [0.485, 0.456, 0.406],
            [0.229, 0.224, 0.225]
        )
    ]),
    'val': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(
            [0.485, 0.456, 0.406],
            [0.229, 0.224, 0.225]
        ])
}
```

A.3 EfficientNet-B6 Initialization

```
device = torch.device(
    'cuda' if torch.cuda.is_available() else 'cpu'
)

model_eff_b6 = models.efficientnet_b6(
    weights='IMAGENET1K_V1'
)

num_ftrs_b6 = model_eff_b6.classifier[1].in_features

model_eff_b6.classifier[1] = nn.Linear(
    num_ftrs_b6,
    len(class_names)
)

model_eff_b6 = model_eff_b6.to(device)
```

A.4 Training Loop

```
def train_model(model, criterion, optimizer, num_epochs=3):
    model.to(device)

    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0
        running_corrects = 0
        total_samples = 0

        for inputs, labels in train_loader:
            inputs = inputs.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()

            outputs = model(inputs)
            loss = criterion(outputs, labels)

            loss.backward()
            optimizer.step()

            _, preds = torch.max(outputs, 1)

            running_loss += (
                loss.item() * inputs.size(0)
            )
            running_corrects += torch.sum(
                preds == labels
            ).item()
            total_samples += labels.size(0)

        epoch_loss = running_loss / total_samples
        epoch_acc = (
            100 * running_corrects / total_samples
        )
```

```
        print(
            f'Epoch {epoch + 1}/{num_epochs} | '
            f'Loss: {epoch_loss:.4f} | '
            f'Training Accuracy: {epoch_acc:.2f}%'
        )
```

A.5 Fine-Tuning

```
optimizer_b6_fine = optim.Adam(
    model_eff_b6.parameters(),
    lr=0.0001
)

train_model(
    model_eff_b6,
    criterion,
    optimizer_b6_fine,
    num_epochs=2
)

new_val_acc = evaluate_model(
    model_eff_b6,
    val_loader
)
```

A.6 Final Test Evaluation

```
test_acc = evaluate_model(
    model_eff_b6,
    test_loader
)

print(
    f'Final Test Accuracy: {test_acc:.2f}%'
)
```

A.7 Classification Report

```
true_labels, predictions = get_predictions(
    model_eff_b6,
    test_loader
)

print(
    classification_report(
        true_labels,
        predictions,
        target_names=class_names,
        digits=4
    )
)
```

A.8 Gradio Prediction Interface

```
inference_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        [0.485, 0.456, 0.406],
        [0.229, 0.224, 0.225]
    )
])

def predict_garbage(image):
    image = image.convert('RGB')

    input_tensor = (
        inference_transform(image)
        .unsqueeze(0)
        .to(device)
    )

    with torch.no_grad():
        output = deployment_model(
            input_tensor
        )
        probabilities = torch.softmax(
            output, dim=1
        )

    predicted_index = torch.argmax(
        probabilities, dim=1
    ).item()

    return class_names[predicted_index]
```

B Complete Notebook Source

The complete extracted Python source of the supplied notebook is included in the accompanying file `AIProjFinal_code.py`. It preserves the notebook's executable code cells in their original order, including data acquisition, preprocessing, training, evaluation, model saving, and deployment code.

C Reproducibility Notes

The supplied notebook was executed in Google Colab and used a CUDA Tesla T4 GPU. The model uses ImageNet-pretrained EfficientNet-B6 weights. Reproducing the exact numerical results may require the same dataset version, preprocessing, random state, software versions, and trained model state used in the original notebook.